

Rapport V1 – SonicWatch

Ali Can CEBI Anisse HAMDI Paul RABIN Lucas LEFEVBRE

16 novembre 2025

Table des matières

1	Introduction	2
2	Installation et mode d'emploi	2
2.1	Pré-requis	2
2.2	Démarrage rapide	2
2.3	Mode manuel	2
3	Méthodologie	3
3.1	LLM et prompt	3
3.2	Modèle spécialiste	3
3.3	MCP	3
3.4	Interface utilisateur	4
4	Développement : difficultés et solutions	4
5	Conclusion	4

1 Introduction

SonicWatch est un assistant audio qui explique les sorties d'un classifieur de sons urbains. Le modèle spécialiste est entraîné sur UrbanSound8. L'utilisateur fournit un fichier WAV, le chatbot déclenche l'inférence puis commente les résultats en français.

Afin de concilier robustesse et flexibilité, nous séparons les rôles : un LLM généraliste (LM Studio + Mistral) assure la couche conversationnelle, un modèle PyTorch calcule les probabilités, un serveur MCP (FastAPI) sert d'intermédiaire et expose les métriques, et deux orchestrateurs (CLI et Gradio) pilotent l'expérience utilisateur.

2 Installation et mode d'emploi

2.1 Pré-requis

Avant de lancer le projet, il faut :

- Python 3.12
- LM Studio installé sur votre PC
- Le checkpoint pré-entraîné `weights/urbansound_cnn.pt`, fourni avec l'archive. Le dataset complet UrbanSound8K n'est nécessaire que pour réentraîner le modèle.

Après décompression de l'archive du projet :

```
cd DeepLearningProject
python3.12 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

2.2 Démarrage rapide

Depuis la racine du dépôt, un script bash peut être exécuté pour automatiser la mise en route :

```
bash setup.sh
```

Il détecte ou crée l'environnement virtuel, installe toutes les dépendances, vérifie la présence de `weights/urbansound_cnn.pt` puis lance le serveur MCP et l'application Gradio en arrière-plan avec la configuration livrée pour l'évaluation.

Ensuite :

1. Ouvrir LM Studio, charger un modèle Instruct (ex. `mistralai/mistral-7b-instruct-v0.3`) et lancer le serveur local sur `http://127.0.0.1:1234`
2. Ouvrir l'URL Gradio affichée dans le terminal (par défaut `http://127.0.0.1:7860`) pour interagir avec SonicWatch.

2.3 Mode manuel

Pour lancer manuellement les services :

- Serveur MCP : `uvicorn mcp_server.server:app --host 127.0.0.1 --port 8000`
- Serveur LLM : ouvrir LM Studio, charger un modèle Instruct et démarrer le serveur (`http://127.0.0.1:1234`)
- Orchestrateurs :

- Terminal : `python chatbot/orchestrator_lmstudio.py`
- Interface Web : `python app_gradio.py`.

L'utilisateur peut saisir une consigne textuelle ("Écoute le son `data/subset/...wav`") ou importer un WAV via l'interface graphique.

3 Méthodologie

3.1 LLM et prompt

LM Studio sert de passerelle compatible OpenAI. Nous y chargeons `mistralai/mistral-7b-instruct-v3`, un modèle instruct léger. Le prompt système (`chatbot/prompt.md`) définit SonicWatch. Les orchestrateurs (`chatbot/orchestrator_lmstudio.py`, `app_gradio.py`) utilisent simplement `openai.OpenAI` avec `base_url=f"{LMSTUDIO_BASE}/v1"` et la clé factice `lm-studio`. Les tours de dialogue `system/user/assistant` sont envoyés via `client.chat.completions.create(...)`, et le LLM ne voit jamais les WAV : il ne reçoit que des résumés JSON ou textuels issus du serveur MCP.

3.2 Modèle spécialiste

UrbanSound8K contient ~ 8732 extraits répartis en 10 classes/folds. Dans `model/` :

- `train.py` : pipeline principal (log-mels via `features.py`, CNN PyTorch, SpecAugment optionnel, `WeightedRandomSampler`, checkpointing)
- `precompute_embeddings.py` et `train_embeddings.py` : pipeline basé sur les embeddings PANNs et une tête linéaire (`weights/linear_head.pkl`)
- `evaluate.py`, `infer.py`, `make_subset.py` : reproduction des expériences, subset léger, inférences unitaires.

Deux variantes sont maintenues :

- 1) **CNN log-mel** : modèle convolutif léger entraîné à partir de zéro, exporté dans `weights/urbansound_cnn.pt` (71–75% d'accuracy, macro-F1 $\approx 0,72$)
- 2) **Baseline embeddings** : projection via PANNs suivie d'un classifieur linéaire/MLP, utilisée comme point de comparaison (rapports "embeddings").

Les métriques (accuracy, macro-F1, par classe) sont exportées dans `reports/metrics*.json` et des matrices de confusion PNG, permettant de recharger les performances sans relancer l'entraînement et de tenir les poids prêts pour le serveur MCP.

3.3 MCP

`mcp_server/server.py` est une API FastAPI qui charge les poids au démarrage, met en cache les modèles et expose :

- GET `/health` : statut, device (CPU/GPU), checkpoint utilisé
- POST `/infer` : prend `wav_path`, `topk`, `use_subset`, renvoie `predicted_label`, `confidence`, `threshold`, `is_confident`, `top_k`
- GET `/metrics?variant={cnn, embeddings}` et GET `/reports` : fournissent les métriques JSON et la liste des rapports.

L'orchestrateur appelle ces endpoints, construit un bloc textuel (classe, confiance, top-k, statistiques globales) et le transmet au LLM, garantissant que les chiffres proviennent du modèle spécialiste.

3.4 Interface utilisateur

`app_gradio.py` fournit l'UI Web. Basée sur `gr.ChatInterface`.

4 Développement : difficultés et solutions

- **Mise en place de l'environnement** : UrbanSound8K est un dataset volumineux et on a eu des difficultés pour trouver les bonnes métriques pour l'entraînement. Nous conservons `scripts/setup_and_run.sh` pour le réentraînement, mais la version livrée repose sur le checkpoint inclus et `setup.sh`, évitant tout téléchargement/entraînement lors de l'évaluation.
- **Conception du modèle** : Pour équilibrer les classes et limiter l'overfitting, nous combinons `WeightedRandomSampler`, `SpecAugment` léger et `early stopping`. La baseline PANNs sert de garde-fou pour détecter les régressions.
- **Intégration LLM** : LM Studio n'accepte que `user/assistant`, nous transformons donc les prompts système en pseudo-messages utilisateurs (`helper prepare_messages_for_lmstudio`).
- **Interface Gradio** : L'ancienne zone d'upload prenait trop d'espace. Nous l'avons remplacée par `UploadButton`, ajouté des exemples guidés et un design `glassmorphism` pour rendre visibles les états et permettre le glisser-déposer de WAV.

5 Conclusion

Cette première version de SonicWatch livre un pipeline complet : modèles CNN et embeddings entraînés, métriques exportées, serveur MCP opérationnel, orchestrateur CLI et interface graphique connectés à un LLM local.

Perspectives d'amélioration :

1. Ajouter de nouvelles classes pour améliorer l'entraînement.
2. Améliorer l'interface et le prompt (messages plus pédagogiques, meilleure gestion des cas ambigus, nouveaux exemples guidés)
3. Simplifier le déploiement (scripts additionnels, conteneurisation, alternative à LM Studio).

Ces améliorations offrent une base solide pour une version 2 plus performante et mieux adaptée.